تاريخ: 10/4/2026

Assignment - 1

Full Name: Zeineddin Meghari

SAP ID: 71651

GitHub Repository Link:

Q1. Algorithm Design and Asymptotic Analysis

Given an array of n integers, design an algorithm to find the largest and smallest elements in the array

1. Algorithmic strategy

The algorithm starts by assuming that the fairt element of the array is both the minimum and maximum value

Then, it goes through the remaining elements one by one.

for each element:

- if the element is greater than the current maximum, update the maximum
- if the element is smaller than the current minimum, update the minimum

2. Pseudocode

Algorithm FindMinMax(A, n)

min A[0]

max A[0]

for i ← 1 to n - 1 do

if A[i] < min then

min A[i]

if A[i] > max then

max ← A[i]

return min, max

3. Major primitive operations

- Assignment operations: • min ← A[0]
  • max ← A[0]
- comparison operations: • A[i] < min
  • A[i] > Max
- increment operation of the loop variable
- Array access: • A[i]

The loop runs n - 1 times

in every iteration, at most two main comparisons are performed

2(n-1)

4. Best_Case, Worst case

- Best-case

Even if the first element is already the minimum or maximum the algorithm still needs to check every element

therefore:

$T(n) = n-1$

best case time complexity is $\Omega(n)$

- Worst case

it performs at most two comparisons for every remaining element

$T(n) = 2(n-1)$

worst case time complexity is: $O(n)$

5. Asymptotic Complexity

Since the algorithm must examine every element regardless of the input arrangement:

Best case = $\Omega(n)$

worst case = $O(n)$

overall = $\Theta(n)$

Therefore running time is:

$\Theta(n)$

6. Space complexity

the algorithm only uses two additional variables:

min

max

it does not create another array or data structure.

therefore, the space complexity is: $O(1)$

تاریخ:

Q2. Design and compare algorithms

Given n integers, design an algorithim to determine whether particular value X exists in the array

Design two Different algorithms / strategies

Algorithm 1: linear search

linear checks every element of the array one by one until the required value X is found.

if algorith ends without finding x, it returns false

pseudocode

```
Algorithm linearSearch(A, n, X)
for i<-0 to n-1
    if A[i] = X
        return true
return false
```

Best case time, the best case happens when X is the first element only one comparison is required

$T(n) = 1$

therefore

Big-O = $O(n)$

Big-omega = $\Omega(1)$

تاریخ: _____

worst time complexity

- X is the last element,
- X does not exist in the array

The algorithm must examine all n elements

$T(n) = n$

worst case = $O(n)$

The general complexity of linear search is:

Best case = $\Omega(1)$ worst case = $O(n)$ Average = $\Theta(n)$

Space Complexity

linear Search uses only a few additional variables

Space Complexity = $O(1)$

Algorithm 2 : Binary Search

Binary search works on a sorted array.

it compares X with the middle element of the array.

if X is equal to the middle element, the value is found

if X is smaller than the middle element, the algorithm continues searching in the left half.

if X is Greater than the middle element, it continuess searching in the right half.

process continues until X is found.

تاریخ: _____

pseudoCode

```
algorithm Binary search
low ← 0
high ← n-1

  while low ≤ high do
      mid ← (low + high) / 2
      if A(mid) = X then
          return true
      else if X < A[mid]
          high ← mid-1
      else
          low ← mid+1

  return false
```

Best Case time

just with one comparison is required

Best Case = $\Omega(1)$

Worst case time complexity

Binnary Search divides the search area into two halves after every comparison

Ex: n
n/2
n/4
n/8
...
1

need to determine how many times n can be divided

$n/2^k = 1$

Therefore:

$n = 2^k$

Taking log base 2:

$k = \log_2(n)$

Therefore:

worst case $= O(\log n)$

The complexity of Binary Search is:

Best case $= \Omega(1)$

worst case $= O(\log n)$

Avearage $= \Theta(\log n)$

Space complexity

Space complexity $= O(1)$

تاریخ: _____

Comparison of linear Search and Binary Search

linear

- works on sorted or un sorted arrays.
- Checks elements one by one.
- Worst-case complexity is O(n)
- Suitable for small or unsorted data

Binary

- Requires sorted array
- Divides the search space into half each time
- worst-Case complexity is O(log n).
- Better for large sorted arrays.

liner = 1000000 comparisons

Binary = 20 comparisons

Effect of input size and arrangement

for small unsorted array, linear search is better

for large and sorted array, Binary search is more efficient

تاریخ: _____ / /

Conclusion

for finding minimum and maximum values:

time complexity = $\theta(n)$

Space complexity = $o!$

for searching

linear Search	Binary Search
Best case = $\Omega(1)$	Best case = $\Omega(1)$
worst case = $o(n)$	worst case = $O(\log n)$
average = $\theta(n)$	average = $\theta(\log n)$

liner search is suitable for small or unsorted data while Binary search is better for large sorted data